Introdução a Maratona e a C++

Escaneie para acessar o slide da Aula 1

luisenilton.github.io/Curso-Maratona/aula1

Sumário da Aula

1. Introdução a Maratona e a C++
2. Sumário da Aula
3. Estrutura básica
4. Tipos de variáveis
5. Entrada e Saida
6. Entrada e saida em numeros reais
7. Caracteres
8. Operadores
9. Auto
10. Overflow
11. Operadores lógicos
12. Condicionais
13. Curto circuito
14. Loops
15. Array
16. String
17. String
18. Vector
19. ForEach
20. Funções
21. Funções úteis de C++
22. Algoritmos básicos
23. Algoritmos básicos
24. Problema
25. Referências
26. Contatos
27. Obrigado por acompanhar a aula

Estrutura básica

```
#include <bits/stdc++.h> // inclusão da biblioteca, que tem tudo que precisamos para rodar

using namespace std; // evita termos que escrever std:: em tudo

int main(){

    cout << "Hello world" << endl;

    return 0;
}
```

Estrutura básica

```
#include <bits/stdc++.h> // inclusão da biblioteca, que tem tudo que precisamos para rodar

using namespace std; // evita termos que escrever std:: em tudo

int main(){

    cout << "Hello world" << endl;

    return 0;
}
```

Estrutura básica

```
#include <bits/stdc++.h> // inclusão da biblioteca, que tem tudo que precisamos para rodar

using namespace std; // evita termos que escrever std:: em tudo

int main(){

    cout << "Hello world" << endl;

    return 0;
}
```

Estrutura básica

```
#include <bits/stdc++.h> // inclusão da biblioteca, que tem tudo que precisamos para rodar

using namespace std; // evita termos que escrever std:: em tudo

int main(){

    cout << "Hello world" << endl;

    return 0;
}
```

Estrutura básica

```
#include <bits/stdc++.h> // inclusão da biblioteca, que tem tudo que precisamos para rodar

using namespace std; // evita termos que escrever std:: em tudo

int main(){

    cout << "Hello world" << endl;

    return 0;
}
```

Estrutura básica

```
#include <bits/stdc++.h> // inclusão da biblioteca, que tem tudo que precisamos para rodar

using namespace std; // evita termos que escrever std:: em tudo

int main(){

    cout << "Hello world" << endl;

    return 0;
}
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Tipos de variáveis

Inteiros

```
int a = 2; // vai aproximadamente de -2 * 10e9 até 2 * 10e9
long long b = 3;// vai aproximadamente de -4 * 10e8 até 4 * 10e18
```

Ponto flutuante

```
double a = 2.1; // vai de  1.7e-308 and 1.7e+308
long double b = 3.2;// vai de 1.18973e+4932 até 3.3621e-4932

// podem ter problemas de precisão na hora de imprimir
```

Booleanos

```
bool a = true;
bool b = false;

// 0 tem valor false, e qualquer outro numero true
bool c = 0, d = 2;
```

Entrada e Saida

lerInt.cpp

```
int x,y;
cin >> x >> y;
cout << x << " " << y << '\n';
cout << x << " " << y << endl;
```

'\n' é apenas quebra de linha, endl libera o buffer.

Existem funções que aceleram a velocidade de entrada e saida do programa

fastInpOut.cpp

```
#define endl '\n'
// Se usarmos endl ele vai liberar o buffer o que gasta muito tempo.
int main(){

  ios::sync_with_stdio(false); // Não é possivel misturar printf com cout depois de rodar essa linha
  cin.tie(NULL)
}
// Ele não limpa mais o buffer toda vez que vai ler um numero
```

Entrada e Saida

lerInt.cpp

```
int x,y;
cin >> x >> y;
cout << x << " " << y << '\n';
cout << x << " " << y << endl;
```

'\n' é apenas quebra de linha, endl libera o buffer.

Existem funções que aceleram a velocidade de entrada e saida do programa

fastInpOut.cpp

```
#define endl '\n'
// Se usarmos endl ele vai liberar o buffer o que gasta muito tempo.
int main(){

  ios::sync_with_stdio(false); // Não é possivel misturar printf com cout depois de rodar essa linha
  cin.tie(NULL)
}
// Ele não limpa mais o buffer toda vez que vai ler um numero
```

Entrada e Saida

lerInt.cpp

```
int x,y;
cin >> x >> y;
cout << x << " " << y << '\n';
cout << x << " " << y << endl;
```

'\n' é apenas quebra de linha, endl libera o buffer.

Existem funções que aceleram a velocidade de entrada e saida do programa

fastInpOut.cpp

```
#define endl '\n'
// Se usarmos endl ele vai liberar o buffer o que gasta muito tempo.
int main(){

  ios::sync_with_stdio(false); // Não é possivel misturar printf com cout depois de rodar essa linha
  cin.tie(NULL)
}
// Ele não limpa mais o buffer toda vez que vai ler um numero
```

Entrada e Saida

lerInt.cpp

```
int x,y;
cin >> x >> y;
cout << x << " " << y << '\n';
cout << x << " " << y << endl;
```

'\n' é apenas quebra de linha, endl libera o buffer.

Existem funções que aceleram a velocidade de entrada e saida do programa

fastInpOut.cpp

```
#define endl '\n'
// Se usarmos endl ele vai liberar o buffer o que gasta muito tempo.
int main(){

  ios::sync_with_stdio(false); // Não é possivel misturar printf com cout depois de rodar essa linha
  cin.tie(NULL)
}
// Ele não limpa mais o buffer toda vez que vai ler um numero
```

Entrada e Saida

lerInt.cpp

```
int x,y;
cin >> x >> y;
cout << x << " " << y << '\n';
cout << x << " " << y << endl;
```

'\n' é apenas quebra de linha, endl libera o buffer.

Existem funções que aceleram a velocidade de entrada e saida do programa

fastInpOut.cpp

```
#define endl '\n'
// Se usarmos endl ele vai liberar o buffer o que gasta muito tempo.
int main(){

  ios::sync_with_stdio(false); // Não é possivel misturar printf com cout depois de rodar essa linha
  cin.tie(NULL)
}
// Ele não limpa mais o buffer toda vez que vai ler um numero
```

Entrada e Saida

lerInt.cpp

```
int x,y;
cin >> x >> y;
cout << x << " " << y << '\n';
cout << x << " " << y << endl;
```

'\n' é apenas quebra de linha, endl libera o buffer.

Existem funções que aceleram a velocidade de entrada e saida do programa

fastInpOut.cpp

```
#define endl '\n'
// Se usarmos endl ele vai liberar o buffer o que gasta muito tempo.
int main(){

  ios::sync_with_stdio(false); // Não é possivel misturar printf com cout depois de rodar essa linha
  cin.tie(NULL)
}
// Ele não limpa mais o buffer toda vez que vai ler um numero
```

Entrada e Saida

lerInt.cpp

```
int x,y;
cin >> x >> y;
cout << x << " " << y << '\n';
cout << x << " " << y << endl;
```

'\n' é apenas quebra de linha, endl libera o buffer.

Existem funções que aceleram a velocidade de entrada e saida do programa

fastInpOut.cpp

```
#define endl '\n'
// Se usarmos endl ele vai liberar o buffer o que gasta muito tempo.
int main(){

  ios::sync_with_stdio(false); // Não é possivel misturar printf com cout depois de rodar essa linha
  cin.tie(NULL)
}
// Ele não limpa mais o buffer toda vez que vai ler um numero
```

Entrada e saida em numeros reais

Imprimir double pode causar erro de precisão por causa de arredondamentos.

```
1  double x,y;
2  cin >> x >> y;
3  double z = x * y;
4  cout << z << endl;
```

Questão Exemplo

OBS: Ler double é muito lento, se possível leia tudo como inteiro e depois converta.

Entrada e saida em numeros reais

Imprimir double pode causar erro de precisão por causa de arredondamentos.

```
// Solução
double x,y;
cin >> x >> y;

cout << setprecision(8) << fixed << z << endl;
```

Questão Exemplo

OBS: Ler double é muito lento, se possível leia tudo como inteiro e depois converta.

Caracteres

lerCarac.cpp

```
char c; cin >> c; // ler somente um caractere
cout << c << endl;

// Caracteres também são o seu valor em ascii
// então você pode somar
char aux = 'a' + 2; // vai ser igual a 'c'
cout << aux << endl;
```

Tabela Ascii

Caracteres

lerCarac.cpp

```
char c; cin >> c; // ler somente um caractere
cout << c << endl;

// Caracteres também são o seu valor em ascii
// então você pode somar
char aux = 'a' + 2; // vai ser igual a 'c'
cout << aux << endl;
```

Tabela Ascii

Caracteres

lerCarac.cpp

```
char c; cin >> c; // ler somente um caractere
cout << c << endl;

// Caracteres também são o seu valor em ascii
// então você pode somar
char aux = 'a' + 2; // vai ser igual a 'c'
cout << aux << endl;
```

Tabela Ascii

Caracteres

lerCarac.cpp

```
char c; cin >> c; // ler somente um caractere
cout << c << endl;

// Caracteres também são o seu valor em ascii
// então você pode somar
char aux = 'a' + 2; // vai ser igual a 'c'
cout << aux << endl;
```

Tabela Ascii

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  int a = 5,b = 2;
2  int teto = a/b;
3  cout << teto << endl; // isso printa 2
```

Operadores

Operadores aritméticos

numericOp.cpp

```
int a = 2,b = 3;
int c = a + b;
int d= a - b;
int e = a * b;
int f = a/b; // divisão inteira (arredonda pra baixo)
int r = a % b; // resto da divisão
```

Problema

Como calcular o teto de uma divisão inteira sem precisar converter para double?

```
1  // Solução
2  int a = 5,b = 2;
3  int teto = (a + b - 1)/b;
4  cout << teto << endl; // 3
```

Auto

Tipo de variável que deduz automaticamente seu tipo com base no que lhe foi atribuido.

```
int a = 2;
auto x = a; // int
auto y = 2.2; // double
auto z = x + y;
auto str = "abcd";
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9; //10^9
a = a * a; // resultado = 10e18
cout << a << endl;
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
1  int a = 1e9; //10^9
2  a = a * a; // resultado = 10e18
3  cout << a << endl;
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9; //10^9
a = a * a; // resultado = 10e18
cout << a << endl;
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9; //10^9
a = a * a; // resultado = 10e18
cout << a << endl;
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
long long a = 1e9;
a = a * a;
cout << a << endl; //10 a 18 cabe no long long
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
long long a = 1e9;
a = a * a;
cout << a << endl; //10 a 18 cabe no long long
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
long long a = 1e9;
a = a * a;
cout << a << endl; //10 a 18 cabe no long long
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
long long a = 1e9;
a = a * a;
cout << a << endl; //10 a 18 cabe no long long
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9;
long long b = (1000) * a ;
cout << b << endl;
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9;
long long b = (1000) * a ;
cout << b << endl;
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9;
long long b = (1000) * a ;
cout << b << endl;
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9;
long long b = (1000) * (long long)a ;
cout << b << endl;
// primeiro a operação é realizada e depois o valor é colocado na variável, então ele não converte.
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9;
long long b = (1000) * (long long)a ;
cout << b << endl;
// primeiro a operação é realizada e depois o valor é colocado na variável, então ele não converte.
```

Overflow

Ocorre quando o resultado de uma operação é maior (ou menor) do que o tamanho suportado pelo tipo

```
int a = 1e9;
long long b = (1000) * (long long)a ;
cout << b << endl;
// primeiro a operação é realizada e depois o valor é colocado na variável, então ele não converte.
```

Operadores lógicos

Operados utilizados na lógica boleana

```
bool a = true, b = false, c = true;

bool op_or = a || b; // retorna true se pelo menos 1 for true

bool op_and = a && b; // retorna true se os dois forem verdadeiros

a = !a; // inverte o valor da condição
```

Operadores lógicos

Operados utilizados na lógica boleana

```
bool a = true, b = false, c = true;

bool op_or = a || b; // retorna true se pelo menos 1 for true

bool op_and = a && b; // retorna true se os dois forem verdadeiros

a = !a; // inverte o valor da condição
```

Operadores lógicos

Operados utilizados na lógica boleana

```
bool a = true, b = false, c = true;

bool op_or = a || b; // retorna true se pelo menos 1 for true

bool op_and = a && b; // retorna true se os dois forem verdadeiros

a = !a; // inverte o valor da condição
```

Operadores lógicos

Operados utilizados na lógica boleana

```
bool a = true, b = false, c = true;

bool op_or = a || b; // retorna true se pelo menos 1 for true

bool op_and = a && b; // retorna true se os dois forem verdadeiros

a = !a; // inverte o valor da condição
```

Operadores lógicos

Operados utilizados na lógica boleana

```
bool a = true, b = false, c = true;

bool op_or = a || b; // retorna true se pelo menos 1 for true

bool op_and = a && b; // retorna true se os dois forem verdadeiros

a = !a; // inverte o valor da condição
```

Condicionais

Utilizaremos apenas de if-else

```
int a,b; cin >> a >> b;

if(a > b){
  // se a for menor que b entra aq
}else if(a< b){
  // se a<b entra aq
}else{
  // se a == b entra aq
}
```

Problema Motivador

Condicionais

Utilizaremos apenas de if-else

```
int a,b; cin >> a >> b;

if(a > b){
  // se a for menor que b entra aq
}else if(a< b){
  // se a<b entra aq
}else{
  // se a == b entra aq
}
```

Problema Motivador

Condicionais

Utilizaremos apenas de if-else

```
int a,b; cin >> a >> b;

if(a > b){
  // se a for menor que b entra aq
}else if(a< b){
  // se a<b entra aq
}else{
  // se a == b entra aq
}
```

Problema Motivador

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
// a expressão a[i] < 4 será avaliada?
if(i < 3 && a[i] < 4){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
// a expressão a[i] < 4 será avaliada?
if(i < 3 && a[i] < 4){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
1  int a[3] ={1,2,3};
2  int i = 3;
3  // a expressão a[i] < 4 será avaliada?
4  if(i < 3 && a[i] < 4){
5
6  }
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
// a expressão a[i] < 4 será avaliada?
if(i < 3 && a[i] < 4){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
/* Não!! como o programa sabe que no && se uma condição for falsa, tudo já é falso,
então a expressão não precisa mais ser avaliada.
*/
if(i < 3 && a[i] < 4){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
1  int a[3] ={1,2,3};
2  int i = 3;
3  /* Não!! como o programa sabe que no && se uma condição for falsa, tudo já é falso,
4  então a expressão não precisa mais ser avaliada.
5  */
6  if(i < 3 && a[i] < 4){
7
8  }
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
/* Não!! como o programa sabe que no && se uma condição for falsa, tudo já é falso,
então a expressão não precisa mais ser avaliada.
*/
if(i < 3 && a[i] < 4){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
// Isso também ocorre no ||
// Se um termo for verdadeiro, tudo já é verdade e a expressão não precisa mais ser avaliada.
if(1 == 1 || i > 5){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
// Isso também ocorre no ||
// Se um termo for verdadeiro, tudo já é verdade e a expressão não precisa mais ser avaliada.
if(1 == 1 || i > 5){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
// Isso também ocorre no ||
// Se um termo for verdadeiro, tudo já é verdade e a expressão não precisa mais ser avaliada.
if(1 == 1 || i > 5){

}
```

Curto circuito

Curto circuito ocorre quando o programa retorna o valor de uma expressão booleana sem avaliar todos os seus termos.

```
int a[3] ={1,2,3};
int i = 3;
// Isso também ocorre no ||
// Se um termo for verdadeiro, tudo já é verdade e a expressão não precisa mais ser avaliada.
if(1 == 1 || i > 5){

}
```

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n--){ // enquanto n for > 0

}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n--){ // enquanto n for > 0

}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n--){ // enquanto n for > 0

}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n--){ // enquanto n for > 0

}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n--){ // enquanto n for > 0

}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n> 0){ // enquanto n for > 0
  n--;
}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n> 0){ // enquanto n for > 0
  n--;
}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Loops

For

```
int n; cin >> n;
// for que roda n vezes, e executa o que está dentro
for(int i = 0;i < n;i++){

}
```

While

```
int n; cin >> n;
// while que roda n vezes, e executa o que está dentro
while(n> 0){ // enquanto n for > 0
  n--;
}
```

Extra

Também existe um foreach em C++, entraremos em detalhes posteriormente.

Array

Estrutura guarda elementos de forma sequencial na memória. É 0 Indexado, os valores começam no indice 0;

```
int a[5] = {0,1,2,3,4};
int c[2]; // se criado vazio, é preenchido com lixo da memória

// pra acessar uma posição individualmente
int x = a[1];

// pode ser criado dinamicamente
int n; cin >> n;
int b[n];

// for que lê todas as posições de um array de tamanho n
for(int i = 0; i < n;i++){
  cin >> b[i];
}
```

Array

Estrutura guarda elementos de forma sequencial na memória. É 0 Indexado, os valores começam no indice 0;

```
int a[5] = {0,1,2,3,4};
int c[2]; // se criado vazio, é preenchido com lixo da memória

// pra acessar uma posição individualmente
int x = a[1];

// pode ser criado dinamicamente
int n; cin >> n;
int b[n];

// for que lê todas as posições de um array de tamanho n
for(int i = 0; i < n;i++){
  cin >> b[i];
}
```

Array

Estrutura guarda elementos de forma sequencial na memória. É 0 Indexado, os valores começam no indice 0;

```
int a[5] = {0,1,2,3,4};
int c[2]; // se criado vazio, é preenchido com lixo da memória

// pra acessar uma posição individualmente
int x = a[1];

// pode ser criado dinamicamente
int n; cin >> n;
int b[n];

// for que lê todas as posições de um array de tamanho n
for(int i = 0; i < n;i++){
  cin >> b[i];
}
```

Array

Estrutura guarda elementos de forma sequencial na memória. É 0 Indexado, os valores começam no indice 0;

```
int a[5] = {0,1,2,3,4};
int c[2]; // se criado vazio, é preenchido com lixo da memória

// pra acessar uma posição individualmente
int x = a[1];

// pode ser criado dinamicamente
int n; cin >> n;
int b[n];

// for que lê todas as posições de um array de tamanho n
for(int i = 0; i < n;i++){
  cin >> b[i];
}
```

Array

Estrutura guarda elementos de forma sequencial na memória. É 0 Indexado, os valores começam no indice 0;

```
int a[5] = {0,1,2,3,4};
int c[2]; // se criado vazio, é preenchido com lixo da memória

// pra acessar uma posição individualmente
int x = a[1];

// pode ser criado dinamicamente
int n; cin >> n;
int b[n];

// for que lê todas as posições de um array de tamanho n
for(int i = 0; i < n;i++){
  cin >> b[i];
}
```

Array

Estrutura guarda elementos de forma sequencial na memória. É 0 Indexado, os valores começam no indice 0;

```
int a[5] = {0,1,2,3,4};
int c[2]; // se criado vazio, é preenchido com lixo da memória

// pra acessar uma posição individualmente
int x = a[1];

// pode ser criado dinamicamente
int n; cin >> n;
int b[n];

// for que lê todas as posições de um array de tamanho n
for(int i = 0; i < n;i++){
  cin >> b[i];
}
```

Array

Estrutura guarda elementos de forma sequencial na memória. É 0 Indexado, os valores começam no indice 0;

```
int a[5] = {0,1,2,3,4};
int c[2]; // se criado vazio, é preenchido com lixo da memória

// pra acessar uma posição individualmente
int x = a[1];

// pode ser criado dinamicamente
int n; cin >> n;
int b[n];

// for que lê todas as posições de um array de tamanho n
for(int i = 0; i < n;i++){
  cin >> b[i];
}
```

String

lerString.cpp

```
string s; // string é um array de caractere
cin >> s;
cout << s << endl;
char c = 'a';
s[0] = c; // uma posição dentro da string é um char
cout << s.size() << endl; // retorna o tamanho da string
s.push_back(c); // adiciona um caractere ao final da string
```

Mais métodos de string na documentação.

String

lerString.cpp

```
string s; // string é um array de caractere
cin >> s;
cout << s << endl;
char c = 'a';
s[0] = c; // uma posição dentro da string é um char
cout << s.size() << endl; // retorna o tamanho da string
s.push_back(c); // adiciona um caractere ao final da string
```

Mais métodos de string na documentação.

String

lerString.cpp

```
string s; // string é um array de caractere
cin >> s;
cout << s << endl;
char c = 'a';
s[0] = c; // uma posição dentro da string é um char
cout << s.size() << endl; // retorna o tamanho da string
s.push_back(c); // adiciona um caractere ao final da string
```

Mais métodos de string na documentação.

String

lerString.cpp

```
string s; // string é um array de caractere
cin >> s;
cout << s << endl;
char c = 'a';
s[0] = c; // uma posição dentro da string é um char
cout << s.size() << endl; // retorna o tamanho da string
s.push_back(c); // adiciona um caractere ao final da string
```

Mais métodos de string na documentação.

String

lerString.cpp

```
string s; // string é um array de caractere
cin >> s;
cout << s << endl;
char c = 'a';
s[0] = c; // uma posição dentro da string é um char
cout << s.size() << endl; // retorna o tamanho da string
s.push_back(c); // adiciona um caractere ao final da string
```

Mais métodos de string na documentação.

String

lerString.cpp

```
string s; // string é um array de caractere
cin >> s;
cout << s << endl;
char c = 'a';
s[0] = c; // uma posição dentro da string é um char
cout << s.size() << endl; // retorna o tamanho da string
s.push_back(c); // adiciona um caractere ao final da string
```

Mais métodos de string na documentação.

String

Para ler uma linha inteiro (inclusive espaços) temos que usar getline.

lerString.cpp

```
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl;
```

Digamos que o formato da entrada seja :

2

linha de texto

```
int x; cin >> x;
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl; // vai imprimir uma quebra de linha
```

String

Para ler uma linha inteiro (inclusive espaços) temos que usar getline.

lerString.cpp

```
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl;
```

Digamos que o formato da entrada seja :

2

linha de texto

```
int x; cin >> x;
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl; // vai imprimir uma quebra de linha
```

String

Para ler uma linha inteiro (inclusive espaços) temos que usar getline.

lerString.cpp

```
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl;
```

Digamos que o formato da entrada seja :

2

linha de texto

```
int x; cin >> x;
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl; // vai imprimir uma quebra de linha
```

String

Para ler uma linha inteiro (inclusive espaços) temos que usar getline.

lerString.cpp

```
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl;
```

Digamos que o formato da entrada seja :

2

linha de texto

```
int x; cin >> x;
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl; // vai imprimir uma quebra de linha
```

String

Para ler uma linha inteiro (inclusive espaços) temos que usar getline.

lerString.cpp

```
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl;
```

Digamos que o formato da entrada seja :

2

linha de texto

```
int x; cin >> x;
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl; // vai imprimir uma quebra de linha
```

String

Para ler uma linha inteiro (inclusive espaços) temos que usar getline.

lerString.cpp

```
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl;
```

Digamos que o formato da entrada seja :

2

linha de texto

```
int x; cin >> x;
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl; // vai imprimir uma quebra de linha
```

String

Para ler uma linha inteiro (inclusive espaços) temos que usar getline.

lerString.cpp

```
string s; // string é um array de caractere
getline(cin, s);
cout << s << endl;
```

Digamos que o formato da entrada seja :

2

linha de texto

```
int x;
string s;
cin >> x;
cin.ignore();
getline(cin, s); // isso vai ler a quebra de linha do cin anterior.
cout << s << endl; // vai imprimir uma quebra de linha
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

Vector

Faz tudo que array faz, mas tem métodos adicionais. Em comparação com o array pode ser um pouco mais lento por ter realocação de memória.

```
vector<int> a(5) = {1,2,3,4,5};

int x = a[1];

int n; cin >> n;
vector<int> b(n);
for(int i = 0; i < n;i++){
  cin >> b[i];
}

// posso adicionar elementos no final, pode causar realocação
b.push_back(5);
// posso retirar elementos do final
b.pop_back();
// .size() retorna o tamanho
cout << b.size() << endl;
```

ForEach

Em C++ existe um for que pode ser utilizado em estruturas como vector e string, para iterar sobre os valores deles sem precisar de índice.

```
vector<int> a(5) = {1,2,3,4,5};

for(int valor: a){
  cout << valor << endl;
}

string s = "abcde";

for(char caractere : s){
  cout << caractere << endl;
}
```

ForEach

Em C++ existe um for que pode ser utilizado em estruturas como vector e string, para iterar sobre os valores deles sem precisar de índice.

```
vector<int> a(5) = {1,2,3,4,5};

for(int valor: a){
  cout << valor << endl;
}

string s = "abcde";

for(char caractere : s){
  cout << caractere << endl;
}
```

ForEach

Em C++ existe um for que pode ser utilizado em estruturas como vector e string, para iterar sobre os valores deles sem precisar de índice.

```
vector<int> a(5) = {1,2,3,4,5};

for(int valor: a){
  cout << valor << endl;
}

string s = "abcde";

for(char caractere : s){
  cout << caractere << endl;
}
```

ForEach

Em C++ existe um for que pode ser utilizado em estruturas como vector e string, para iterar sobre os valores deles sem precisar de índice.

```
vector<int> a(5) = {1,2,3,4,5};

for(int valor: a){
  cout << valor << endl;
}

string s = "abcde";

for(char caractere : s){
  cout << caractere << endl;
}
```

ForEach

Em C++ existe um for que pode ser utilizado em estruturas como vector e string, para iterar sobre os valores deles sem precisar de índice.

```
vector<int> a(5) = {1,2,3,4,5};

for(int valor: a){
  cout << valor << endl;
}

string s = "abcde";

for(char caractere : s){
  cout << caractere << endl;
}
```

ForEach

Em C++ existe um for que pode ser utilizado em estruturas como vector e string, para iterar sobre os valores deles sem precisar de índice.

```
vector<int> a(5) = {1,2,3,4,5};

for(int valor: a){
  cout << valor << endl;
}

string s = "abcde";

for(char caractere : s){
  cout << caractere << endl;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por cópia
int soma(int a,int b){
  int resposta = a+b;
  return resposta;
}


int main(){
  int a = 2,b = 3;
  a = soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por cópia
int soma(int a,int b){
  int resposta = a+b;
  return resposta;
}


int main(){
  int a = 2,b = 3;
  a = soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por cópia
int soma(int a,int b){
  int resposta = a+b;
  return resposta;
}


int main(){
  int a = 2,b = 3;
  a = soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por cópia
int soma(int a,int b){
  int resposta = a+b;
  return resposta;
}


int main(){
  int a = 2,b = 3;
  a = soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por cópia
int soma(int a,int b){
  int resposta = a+b;
  return resposta;
}


int main(){
  int a = 2,b = 3;
  a = soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por referência
// é preciso colocar um &
void soma(int &a,int b){
    a += b;
}


int main(){
  int a = 2,b = 3;
  soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por referência
// é preciso colocar um &
void soma(int &a,int b){
    a += b;
}


int main(){
  int a = 2,b = 3;
  soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por referência
// é preciso colocar um &
void soma(int &a,int b){
    a += b;
}


int main(){
  int a = 2,b = 3;
  soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções

A função tem parâmetros, tipos de retorno e tipo dos parâmetros definidos. A função pode ser void.

Os parâmetros podem ser passados por cópia ou referência, caso sejam passados por referência, se forem alterados dentro da função também serão alterados onde foi chamado.

```
// Passando a por referência
// é preciso colocar um &
void soma(int &a,int b){
    a += b;
}


int main(){
  int a = 2,b = 3;
  soma(a,b);
  cout << a << endl;
  return 0;
}
```

Funções úteis de C++

Min

Recebe 2 valores do mesmo tipo e retorna o menor deles.

```
int a = 3,b = 2;
cout << min(a,b) << endl;
```

Max

Recebe 2 valores do mesmo tipo e retorna o maior deles.

```
int a = 3,b = 2;
cout << max(a,b) << endl;
```

Algoritmos básicos

Achar o maximo num array

```
vector<int> arr = {2,4,5,3,1};
int maxi = 0;
for(auto x : arr){
  if(x > maxi) maxi = x;
}
```

Algoritmos básicos

Achar o maximo num array

```
vector<int> arr = {2,4,5,3,1};
int maxi = 0;
for(auto x : arr){
  maxi = max(maxi,x);
}
```

Algoritmos básicos

Achar o maximo num array

```
vector<int> arr = {2,4,5,3,1};
// pra achar o minimo é a mesma coisa, so que inicialmente tem valor infinito
int mini = INF;
for(auto x : arr){
  mini = min(mini,x);
}
```

Algoritmos básicos

Contar a frequencia de cada valor num array

```
vector<int> arr = {2,4,5,3,1};
vector<int> cnt(maximo + 1);
for(auto x : arr){
  cnt[x]++;
}
```

Algoritmos básicos

Contar a frequencia de cada valor num array

```
string s = "abcd";
// contar a frequencia de cada caractere
vector<int> cnt(26); // a posição 0 guarda pra 'a''e a 25 pra 'z'
for(auto c : s){
  cnt[c - 'a']++;
}
```

Problema

Eu tenho duas strings s e t, como saber se uma é anagrama da outra?

Problema

Referências

- Documentação C++
- CP Algorithms : Site com ótimas explicações para algoritmos mais complexos.
- Curso UFMG : Várias ideias do nosso curso são baseadas nele.
- MaratonUSP : Canal no youtube com aulas muito boas.
- Lista de questões por tópico: Ótimo site (em inglês) que tem lista de questões para quase todos os assuntos.

Contatos

Entre no discord para dúvidas

Entre no grupo de questões

Obrigado por acompanhar a aula

Por favor preencha o formulário de feedback com sugestões e críticas para a próxima aula 🙃.